AudioSurvey AI — Complete Study Guide for Presentation

Version 1.1.0 | March 2026 | Comprehensive Code-Level Walkthrough

1. PROJECT OVERVIEW

What Is AudioSurvey AI?

AudioSurvey AI is an **AI-powered multilingual Interactive Voice Response (IVR) survey platform** that conducts automated telephone-based research surveys. It targets **African refugees who speak Kiswahili**, enabling researchers to collect structured survey responses at scale without in-person enumerators.

Core Idea

Instead of sending a person to interview participants face-to-face, the system **places automated phone calls**, asks questions in **Kiswahili using AI-generated voice**, collects responses (voice or keypad presses), and then uses an **AI/ML pipeline** to process and translate everything to English.

Key Capabilities

Feature	How It Works
Automated Outbound Calling	Scheduler dials participants via Twilio at scheduled times
Inbound Call Support	Participants can call in and complete the survey
Multilingual IVR Prompts	Azure Neural TTS speaks questions in Kiswahili
4 Question Types	INFO (instruction), OPEN (speech), MCQ (keypad), MCQO (keypad + "Other" speech)
Full Call Recording	Entire call recorded (up to 30 min) via Twilio
AI Noise Removal	DeepFilterNet (PyTorch) cleans background noise
AI Transcription	OpenAI Whisper large-v3 converts Kiswahili speech to text
Machine Translation	Google Translate converts Kiswahili text to English
English Audio Generation	Google TTS creates English audio from translations
Excel Data Export	Structured MCQ/MCQO responses exported to spreadsheets
Admin Dashboard	Web UI for managing participants, scheduling, monitoring
Conference Calling	Three-way calls for researcher-moderated interviews

2. TECHNOLOGY STACK

Programming Language

- **Python 3.x** — the entire application is written in Python

Web Framework

- **Flask 3.1.2** — lightweight web framework that handles all HTTP endpoints (webhooks from Twilio, admin dashboard, API routes)

Telephony (Phone Calls)

- **Twilio Voice API** — cloud service that places and receives phone calls
- **TwiML (Twilio Markup Language)** — XML-based language to script what happens during a call (play audio, gather input, record, hangup)

AI/ML Models

- **OpenAI Whisper (large-v3)** — state-of-the-art speech recognition model. Transcribes Kiswahili audio to text. Runs locally on the machine (not cloud API)
- **DeepFilterNet** — PyTorch-based deep learning model for removing background noise from audio recordings
- **Azure Cognitive Services (Neural TTS)** — Microsoft's cloud service for generating natural-sounding Kiswahili voice prompts using SSML (Speech Synthesis Markup Language)
- **Google TTS (gTTS)** — Google's text-to-speech for generating English audio output
- **Google Translate (googletrans)** — translates Kiswahili text to English via scraping

Audio Processing

- **FFmpeg** — command-line tool for audio format conversion and resampling
- **PyTorch + torchaudio** — ML framework powering DeepFilterNet
- **NumPy** — numerical computing for audio data manipulation
- **pydub** — Python audio manipulation library

Data & Export

- **pandas** — data manipulation and analysis
- **openpyxl** — Excel file (.xlsx) generation
- **PyYAML** — YAML configuration file parsing

Infrastructure

- **ngrok** — creates secure HTTPS tunnels so Twilio can reach the local Flask server
- **gunicorn** — production WSGI server

Security

- **werkzeug** — PBKDF2 password hashing for admin authentication
- **python-dotenv** — loads API keys from `.env` file

---

3. PROJECT STRUCTURE (Every File Explained)

```
audiosurvey_ai/
|
|-- run_app.py                    # ENTRY POINT: Starts ngrok + Flask server + opens browse
|-- main.py                       # Standalone batch processor (transcribe/translate/TTS of
|-- config.yaml                   # All settings: Twilio, IVR, audio processing, auth
|-- .env                          # Secret API keys (Twilio, Azure, etc.)
|-- requirements.txt              # 146 Python package dependencies
|
|-- app/                          # Main application code
|   |-- __init__.py               # Makes 'app' a Python package
|   |-- twilio_handler.py         # CORE: Flask app + all webhook routes (1,339 lines)
|   |-- dashboard.py              # Admin web UI (Flask Blueprint)
|   |-- state.py                  # Participant state management (JSON persistence)
|   |-- background_worker.py      # ML processing pipeline (continuous worker thread)
|   |-- scheduler.py              # Automated call scheduler (background thread)
|   |-- audio_preprocess.py       # DeepFilterNet noise removal pipeline
|   |-- transcribe.py             # OpenAI Whisper speech-to-text
|   |-- translate.py              # Google Translate with chunking + retry
|   |-- tts.py                    # Google TTS (English audio output)
|   |-- azure_tts.py              # Azure Neural TTS (IVR Kiswahili prompts)
|   |-- export_excel.py           # Survey response export to Excel
|   |-- utils.py                  # Scheduling helper (time conversion)
|   |-- twilio_utils.py           # Twilio call helpers
|   |-- file_naming.py            # Safe filename generation
|   |-- logger.py                 # Colored console logging
|   |-- auth.py                   # Authentication module
|   |-- runtime_warnings.py       # Suppresses noisy library warnings
|
|-- data/                         # All data storage (file-based, no database)
|   |-- state/
|   |   |-- participants.json     # Main state file: all participant data + responses
|   |   |-- call_log.csv          # Audit log of every call event
|   |   |-- settings.json         # Global settings (paused flag)
|   |-- questions.txt             # Survey questions (76 questions, pipe-delimited format)
|   |-- contacts.csv              # Phone contact list template
|   |-- audio/                    # Raw call recordings (.wav)
|   |-- audio_processed/          # Cleaned audio after DeepFilterNet (.wav)
|   |-- transcripts/              # Whisper transcriptions (.txt, Kiswahili)
|   |-- translations/             # Google Translate output (.txt, English)
|   |-- english_audio/            # gTTS English audio (.mp3)
|   |-- ivr_audio/                # Cached Azure TTS prompts (.mp3)
|   |-- results/                  # Excel exports (.xlsx)
|
|-- packaging/
    |-- macos_dmg/                # macOS .app bundle and DMG installer
```

4. HOW THE APPLICATION STARTS (Entry Points)

Mode 1: Normal Launch — `run_app.py`

When you run `python3 run_app.py`, here is exactly what happens:

```
# Step 1: Start ngrok tunnel
ngrok_proc = subprocess.Popen(["ngrok", "http", "5050"])
# ngrok creates an HTTPS URL like: https://armlike-unadduced-ai.ngrok-free.dev
# This is needed because Twilio requires HTTPS to send webhooks

# Step 2: Fetch the tunnel URL
# Queries ngrok's local API at http://127.0.0.1:4040/api/tunnels
# Extracts the HTTPS public URL

# Step 3: Set environment variable
os.environ["PUBLIC_BASE_URL"] = "https://armlike-unadduced-ai.ngrok-free.dev"

# Step 4: Launch Flask server as subprocess
subprocess.Popen([sys.executable, "-m", "app.twilio_handler", "serve"])
# This runs: python3 -m app.twilio_handler serve

# Step 5: Open browser to admin dashboard
webbrowser.open("http://127.0.0.1:5050/admin")
```

Mode 2: Direct Server — `python3 -m app.twilio_handler serve`

```
# Step 1: Start background services
start_background_services()
#   -> Starts Scheduler thread (daemon, polls every 15 seconds)
#   -> Starts ML Worker thread (daemon, polls every 5 seconds)

# Step 2: Run Flask web server
app.run(host="0.0.0.0", port=5050, debug=False, use_reloader=False)
# Listens on all network interfaces, port 5050
```

Mode 3: Batch Processing — `main.py`

Processes audio files offline (no calls needed):

```
# 1. Transcribe all audio files in data/audio/ using Whisper
# 2. Translate all transcripts from Kiswahili to English
# 3. Generate English audio from translations
```

---

5. CORE APPLICATION — `twilio_handler.py` (Line-by-Line Explanation)

This is the biggest and most important file (1,339 lines). Here's every section:

5.1 Imports & Configuration (Lines 1-127)

```
# Load environment variables from .env file
from dotenv import load_dotenv
load_dotenv()

# Load config.yaml for IVR settings
def load_config(path="config.yaml"):
    with open(path, "r", encoding="utf-8") as f:
        return yaml.safe_load(f) or {}

cfg = load_config()

# Read Twilio credentials from environment
TWILIO_SID = os.getenv("TWILIO_ACCOUNT_SID")      # Account identifier
TWILIO_TOKEN = os.getenv("TWILIO_AUTH_TOKEN")       # Secret token
TWILIO_FROM = os.getenv("TWILIO_FROM_NUMBER")       # Phone number to call from
PUBLIC_BASE_URL = os.getenv("PUBLIC_BASE_URL")       # ngrok HTTPS URL

# IVR configuration from config.yaml
GATHER_TIMEOUT = 6        # Wait 6 seconds for speech/keypad input
SPEECH_TIMEOUT = "auto"   # Auto-detect end of speech
SPEECH_LANGUAGE = "sw-KE" # Kiswahili (Kenya) for speech recognition
RECORDING_MAX_SEC = 1800  # 30 minutes max recording

# Create output directories
for d in [AUDIO_DIR, TRANSCRIPTS_DIR, TRANSLATIONS_DIR, EN_AUDIO_DIR, IVR_AUDIO_DIR]:
    os.makedirs(d, exist_ok=True)
```

5.2 Azure TTS for IVR Prompts (Lines 129-244)

Purpose: Generate Kiswahili voice audio for survey questions played during calls.

```
# Azure credentials
AZURE_SPEECH_KEY = os.getenv("AZURE_SPEECH_KEY")
AZURE_SPEECH_REGION = os.getenv("AZURE_SPEECH_REGION")  # "eastus"
AZURE_TTS_VOICE_SW = "sw-KE-ZuriNeural"   # Kiswahili voice
AZURE_TTS_VOICE_EN = "en-US-JennyNeural"   # English voice

def azure_tts_to_file(text, out_path, voice):
    """Generate audio file from text using Azure Neural TTS"""

    # Escape XML special characters for SSML
    safe_text = text.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;")

    # Configure Azure Speech SDK
    speech_config = speechsdk.SpeechConfig(subscription=AZURE_SPEECH_KEY, region=AZURE_SPEE
    audio_config = speechsdk.audio.AudioOutputConfig(filename=out_path)
    synthesizer = speechsdk.SpeechSynthesizer(speech_config=speech_config, audio_config=auc

    # Build SSML (Speech Synthesis Markup Language) with slower speech rate
    ssml = f"""
    <speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="sw-KE">
        <voice name="{voice}">
            <prosody rate="-15%">    <!-- 15% slower for clarity -->
                {safe_text}
            </prosody>
        </voice>
    </speak>"""

    # Synthesize audio
    result = synthesizer.speak_ssml_async(ssml).get()

def get_prompt_audio_url(text, lang):
    """Returns a PUBLIC URL to a cached MP3 of the prompt"""

    voice = AZURE_TTS_VOICE_SW if lang.startswith("sw") else AZURE_TTS_VOICE_EN

    # Create hash-based filename for caching (same text = same file)
    key = hashlib.sha1(f"{voice}|{format}|{text}".encode("utf-8")).hexdigest()
    filename = f"{key}.mp3"
    out_path = os.path.join(IVR_AUDIO_DIR, filename)

    # Only generate if not already cached
    if not os.path.exists(out_path) or os.path.getsize(out_path) < 2000:
        azure_tts_to_file(text, out_path, voice)

    return f"{PUBLIC_BASE_URL}/ivr-audio/{filename}"
```

Key Concept: TTS audio is **cached by text hash**. If the same question text is asked again, it serves the cached file instead of calling Azure again. This saves money and time.

5.3 Authentication System (Lines 248-482)

```
# Brute-force protection settings
AUTH_MAX_FAILS = 7           # Max failed attempts before lockout
AUTH_LOCK_SECONDS = 900      # 15-minute lockout duration
AUTH_WINDOW_SECONDS = 600    # 10-minute window for counting failures

# Auth state stored in JSON file
def _load_auth_state():
    # Returns: {"fails": {"username|ip": [timestamps]}, "locks": {"username|ip": lock_until

def _record_fail(username):
    """Track failed login attempt"""
    # 1. Load auth state
    # 2. Remove failures older than 10 minutes
    # 3. Add current timestamp
    # 4. If failures >= 7, lock for 15 minutes
    # 5. Save auth state

def _verify_user(users, username, password):
    """Check password against PBKDF2 hash stored in config.yaml"""
    return check_password_hash(user["password_hash"], password)

# Login route
@app.route("/login", methods=["GET", "POST"])
def login_route():
    # GET: Show login page (HTML with CSS inline)
    # POST: Validate credentials
    #   1. Check if account is locked -> show "Try again in X seconds"
    #   2. Verify password hash -> if fail, record failure
    #   3. On success: clear failures, create session (8-hour lifetime)
    #   4. Redirect to /admin
```

Security Features: - Passwords stored as PBKDF2 hashes with SHA-256 (1 million iterations) - Brute-force protection: 7 failures in 10 min = 15 min lockout - Session cookies: HTTPS-only, HttpOnly, SameSite=Lax - Auth events logged to `data/auth_log.jsonl`

5.4 Flask App Setup (Lines 487-523)

```
app = Flask(__name__)
app.register_blueprint(dashboard_bp)  # Admin dashboard routes
app.secret_key = os.getenv("FLASK_SECRET_KEY", "CHANGE_ME_NOW")

# Session security settings
app.config.update(
    SESSION_COOKIE_HTTPONLY=True,      # JavaScript can't read cookies
    SESSION_COOKIE_SAMESITE="Lax",    # CSRF protection
    SESSION_COOKIE_SECURE=True,        # HTTPS only (via ngrok)
    PERMANENT_SESSION_LIFETIME=timedelta(hours=8),  # 8-hour sessions
)

# Route guard: protect admin routes
@app.before_request
def _guard_admin_routes():
    # Public routes (no auth needed): /voice, /start, /next, /recording-done, etc.
    # These are Twilio webhook callbacks - must be publicly accessible

    # Admin routes (/admin/*): require login session or ADMIN_TOKEN
    if path.startswith("/admin"):
        if not _is_logged_in():
            return redirect("/login")
```

5.5 Conference Call Feature (Lines 573-710)

```
@app.route("/admin/conference_call", methods=["POST"])
def admin_conference_call():
    """Create a 3-way conference call between two phone numbers"""

    room = "CONF_" + datetime.utcnow().strftime("%Y%m%d_%H%M%S")

    # Call person 1 -> join conference as host (plays IVR while waiting)
    client.calls.create(to=n1, from_=TWILIO_FROM, url=f"{BASE_URL}/conference_host?room={ro

    # Call person 2 -> join same conference room
    client.calls.create(to=n2, from_=TWILIO_FROM, url=f"{BASE_URL}/conference_join?room={ro

# Host hears IVR survey questions while waiting for moderator to join
@app.route("/conference_ivr", methods=["POST"])
def conference_ivr():
    # Play intro in Kiswahili, then loop through questions
    # When all questions done, start the conference (both parties can talk)
```

5.6 Question Loading & Helper Functions (Lines 715-795)

```
def load_structured_questions():
    """Parse questions.txt into structured list of dictionaries"""
    # Each line: TYPE|Question Text|Option1|Option2|...
    # Returns: [{"type": "mcq", "question": "...", "options": [...]}, ...]

def looks_like_real_speech(s):
    """Filter out silence/noise detected as 'speech'"""
    # Returns False for: empty, "...", "silence", "no speech"
    # Returns True for: actual speech content

def find_participant_by_callsid(state, call_sid):
    """Find which participant a Twilio CallSid belongs to"""
    # Loops through all participants, matches last_call_sid

def get_mcqo_other_digit(question):
    """Find which digit corresponds to 'Other'/'Nyingine' option"""
    # For MCQO questions, identifies the "Other" option number
```

5.7 IVR Call Flow Routes (Lines 809-1016) — THE HEART OF THE SYSTEM

Route 1: `/voice` — Call Starts Here (Line 809)

```
@app.route("/voice", methods=["POST"])
def voice():
    """Called by Twilio when participant answers the phone"""

    xml = """
    <Response>
      <!-- Start recording the entire call (up to 30 minutes) -->
      <Start>
        <Recording maxLength="1800"
                   recordingStatusCallback="/recording-done"
                   recordingStatusCallbackEvent="completed" />
      </Start>

      <!-- Immediately redirect to start the survey -->
      <Redirect>/start</Redirect>
    </Response>"""
```

What happens: The moment someone answers, Twilio starts recording and redirects to `/start`.

Route 2: `/start` — Survey Begins (Line 832)

```
@app.route("/start", methods=["POST"])
def start():
    """Play intro sections and first question"""

    questions = load_structured_questions()  # Load all 76 questions

    # Find which participant this call belongs to
    call_sid = request.values.get("CallSid")
    pid, p = find_participant_by_callsid(state, call_sid)

    # Initialize response storage for this participant
    state[pid]["responses"] = {}
    state[pid]["survey_q_counter"] = 0

    # Play first 2 questions (INFO type = introduction sections)
    # questions[0] = "Habari! Karibu kwenye utafiti..."
    # questions[1] = "Maswali kuhusu Video ya MADO..."
    intro_xml = ""
    for i in range(2):
        intro_url = get_prompt_audio_url(questions[i]["question"], "sw")
        intro_xml += f"<Play>{intro_url}</Play><Pause length='1'/>"

    # Play question 3 (first real question, usually OPEN type)
    # Use <Gather> to capture speech response
    xml = f"""
    <Response>
      {intro_xml}
      <Gather input="speech" timeout="6" speechTimeout="auto"
              action="/next?q=3">
          <Play>{question_audio_url}</Play>
      </Gather>
      <Redirect>/next?q=3</Redirect>  <!-- If no speech detected, still advance -->
    </Response>"""
```

What happens: Plays 2 intro sections, then the first real question with a 6-second speech capture window.

Route 3: `/next` — Main Question Loop (Line 891)

```
@app.route("/next", methods=["POST", "GET"])
def next_question():
    """Process previous answer and play next question"""

    q = int(request.args.get("q", "0"))  # Current question index
    speech = request.values.get("SpeechResult", "").strip()  # Speech from previous questio

    # --- Store OPEN (speech) answer from previous question ---
    if pid and looks_like_real_speech(speech):
        prev_q = q - 1
        if questions[prev_q]["type"] == "open":
            state[pid]["survey_q_counter"] += 1
            survey_q = state[pid]["survey_q_counter"]
            state[pid]["responses"][f"q{survey_q}"] = speech  # Store Kiswahili speech
            # Mark participant as "engaged" (they actually spoke)
            mark_engaged(state, pid)

    # --- Check if survey is complete ---
    if q >= len(questions):
        bye_url = get_prompt_audio_url("Kwaheri.", "sw")  # "Goodbye" in Kiswahili
        return twiml(f"""<Response><Play>{bye_url}</Play><Hangup/></Response>""")

    question = questions[q]

    # --- MCQ or MCQO: Gather keypad digit ---
    if question["type"] in ["mcq", "mcqo"]:
        # Build prompt: "Question text. Press 1 for Option A. Press 2 for Option B..."
        options_text = ""
        for i, opt in enumerate(question["options"], start=1):
            options_text += f"finya {i} kwa {opt}. "  # "press 1 for ..."

        full_q = f"{question['question']}. {options_text}"
        q_url = get_prompt_audio_url(full_q, "sw")

        return twiml(f"""
        <Response>
          <Gather input="dtmf" numDigits="1" timeout="6"
                  action="/mcq-handler?q={q}">
              <Play>{q_url}</Play>
          </Gather>
          <Redirect>/next?q={q}</Redirect>  <!-- Replay if no input -->
        </Response>""")

    # --- INFO: Just play and advance ---
    elif question["type"] == "info":
        q_url = get_prompt_audio_url(question["question"], "sw")
        return twiml(f"""
        <Response>
          <Play>{q_url}</Play>
          <Pause length="1"/>
          <Redirect>/next?q={q+1}</Redirect>
        </Response>""")

    # --- OPEN: Gather speech ---
```

```
    else:
        q_url = get_prompt_audio_url(question["question"], "sw")
        return twiml(f"""
        <Response>
          <Gather input="speech" timeout="6" speechTimeout="auto"
                  action="/next?q={q+1}">
              <Play>{q_url}</Play>
          </Gather>
          <Redirect>/next?q={q+1}</Redirect>
        </Response>""")
```

What happens: This is the question loop. For each question, it determines the type and generates appropriate TwiML. Speech responses go to `/next`, keypad presses go to `/mcq-handler`.

Route 4: `/mcq-handler` — Process Keypad Input (Line 1018)

```
@app.route("/mcq-handler", methods=["POST"])
def mcq_handler():
    """Handle MCQ/MCQO digit press"""

    q = int(request.args.get("q", "0"))
    digit = request.form.get("Digits", "")  # e.g., "1", "2", "3"

    question = questions[q]

    # Store the digit response
    state[pid]["survey_q_counter"] += 1
    survey_q = state[pid]["survey_q_counter"]
    state[pid]["responses"][f"q{survey_q}"] = digit  # Store: "q5" -> "2"

    # For MCQO: check if "Other" was selected
    if question["type"] == "mcqo":
        other_digit = get_mcqo_other_digit(question)  # e.g., "3" for "Nyingine"

        if digit == other_digit:
            # "Other" selected -> collect speech explanation
            prompt = "Umechagua nyingine. Tafadhali sema jibu lako sasa."
            # "You chose other. Please say your answer now."
            prompt_url = get_prompt_audio_url(prompt, "sw")

            return twiml(f"""
            <Response>
              <Play>{prompt_url}</Play>
              <Gather input="speech" timeout="4" action="/mcqo-other-handler?q={q}">
              </Gather>
              <Redirect>/next?q={q+1}</Redirect>
            </Response>""")

    # Normal MCQ or non-"Other" MCQO -> advance to next question
    return twiml(f"""<Response><Redirect>/next?q={q+1}</Redirect></Response>""")
```

Route 5: `/recording-done` — Call Ended, Process Recording (Line 1205)

```
@app.route("/recording-done", methods=["POST"])
def recording_done():
    """Twilio calls this when the call recording is finished"""

    call_sid = request.form.get("CallSid")
    recording_url = request.form.get("RecordingUrl")

    # Find which participant this was
    participant_id, p = find_participant_by_callsid(state, call_sid)
    engaged = p.get("engaged", False)  # Did they actually speak?

    # Download the WAV recording from Twilio
    wav_url = recording_url + ".wav"
    r = requests.get(wav_url, auth=(TWILIO_SID, TWILIO_TOKEN), timeout=60)
    with open(audio_path, "wb") as f:
        f.write(r.content)  # Save to data/audio/participant_001_20260321_143000.wav

    # Queue for ML processing (only if participant engaged)
    if engaged:
        state[participant_id]["processing_status"] = "pending"
        # Background worker will pick this up
    else:
        state[participant_id]["processing_status"] = "saved_no_engagement"

    # Log the event
    log_call_event({...})
```

---

6. STATE MANAGEMENT — `state.py` (How Data Is Stored)

No Database — Everything in JSON Files

The system uses **file-based persistence** instead of a traditional database:

```
STATE_DIR = "data/state"
PARTICIPANTS_PATH = "data/state/participants.json"
CALL_LOG_PATH = "data/state/call_log.csv"
SETTINGS_PATH = "data/state/settings.json"

# Thread safety: RLock allows same thread to acquire lock multiple times
STATE_IO_LOCK = threading.RLock()
```

Participant Data Model

Each participant record in `participants.json`:

```
{
  "participant_001": {
    "phone_e164": "+1234567890",
    "status": "completed",           // pending | in_progress | completed | failed
    "attempts": 2,                   // How many times we called them (max 3)
    "last_call_time": "2026-03-21T14:30:00",
    "last_call_sid": "CA123abc...",  // Twilio's unique call identifier
    "last_call_status": "completed", // Twilio callback status
    "engaged": true,                 // Did they actually speak?
    "last_recording_url": "https://api.twilio.com/...",
    "processing_status": "completed", // pending | processing | completed | failed
    "scheduled_time_utc": "2026-03-21T18:30:00Z",
    "responses": {
      "q1": "Jina langu ni Amina",   // OPEN: Kiswahili speech text
      "q2": "2",                      // MCQ: digit pressed
      "q3": "1",                      // MCQO: digit pressed
      "survey_q_counter": 15          // Total questions answered
    },
    "last_outputs": {
      "audio_path": "data/audio/participant_001_20260321_143000.wav",
      "processed_audio_path": "data/audio_processed/participant_001_20260321_143000.wav",
      "transcript_path": "data/transcripts/participant_001_20260321_143000.txt",
      "translation_path": "data/translations/participant_001_20260321_143000.txt",
      "english_audio_path": "data/english_audio/participant_001_20260321_143000.mp3"
    }
  }
}
```

Thread-Safe File I/O

```
def save_participants(state):
    with STATE_IO_LOCK:  # Only one thread can write at a time
        # Write to temporary file first
        tmp_path = f"{PARTICIPANTS_PATH}.{threading.get_ident()}.tmp"
        with open(tmp_path, "w", encoding="utf-8") as f:
            json.dump(state, f, ensure_ascii=False, indent=2)
        # Atomic replace: prevents corrupted files if crash mid-write
        os.replace(tmp_path, PARTICIPANTS_PATH)
```

Call Eligibility Logic

```
def can_call(state, participant_id, force=False):
    """Determine if we should call this participant now"""

    p = state.get(participant_id)

    # Rule 1: Never call completed or failed participants
    if p["status"] in {"completed", "failed"}:
        return False

    # Rule 2: Max 3 call attempts
    if p["attempts"] >= 3:
        return False

    # Rule 3: Force mode skips scheduling checks
    if force:
        return True

    # Rule 4: Must have a scheduled time that has passed
    if datetime.now() < scheduled_time:
        return False

    # Rule 5: Wait at least 1 hour between retry attempts
    if (now - last_call_time) < timedelta(hours=1):
        return False

    return True
```

---

7. CALL SCHEDULER — `scheduler.py` (How Calls Are Automatically Placed)

```
def run_once(force=False):
    """One scheduler tick - check all participants and call eligible ones"""

    if is_paused() and not force:
        return  # Admin paused the scheduler

    client = Client(TWILIO_SID, TWILIO_TOKEN)  # Twilio API client
    state = load_participants()

    for participant_id, p in state.items():
        phone = p.get("phone_e164")

        if not can_call(state, participant_id, force=force):
            continue

        # Place the call via Twilio API
        call = client.calls.create(
            to=phone,                                    # Participant's phone
            from_=TWILIO_FROM,                           # Our Twilio number
            url=f"{PUBLIC_BASE_URL}/voice",              # Webhook when answered
            record=True,                                 # Record the call
            recording_status_callback=f"{PUBLIC_BASE_URL}/recording-done",
            status_callback=f"{PUBLIC_BASE_URL}/call-status",
            status_callback_event=["completed", "no-answer", "busy", "failed", "canceled"],
        )

        mark_call_started(state, participant_id, call.sid)

def start_scheduler_in_background(interval_sec=15):
    """Start scheduler as daemon thread (runs forever)"""
    def _loop():
        time.sleep(15)  # Wait before first tick
        while True:
            run_once(force=False)
            time.sleep(15)  # Check every 15 seconds

    t = threading.Thread(target=_loop, daemon=True)
    t.start()  # Daemon thread dies when main process exits
```

8. BACKGROUND ML WORKER — `background_worker.py` (AI Processing Pipeline)

This runs as a **daemon thread**, continuously checking for new recordings to process.

```
def process_pending_recordings():
    """Continuous loop: find recordings with status='pending' and process them"""

    while True:
        state = load_participants()

        for pid, p in state.items():
            if p.get("processing_status") != "pending":
                continue  # Skip non-pending participants

            audio_path = p.get("audio_path")
            state[pid]["processing_status"] = "processing"
            save_participants(state)

            # === STAGE 1: DeepFilterNet Noise Removal ===
            processed_audio_path = preprocess_recording(audio_path)
            # Input:  data/audio/participant_001.wav (noisy)
            # Output: data/audio_processed/participant_001.wav (clean)

            # === STAGE 2: Whisper Transcription ===
            text, detected_lang = transcribe_audio(processed_audio_path)
            with open(transcript_path, "w") as f:
                f.write(text)  # Kiswahili text

            # === STAGE 3: Google Translation ===
            if detected_lang == "en":
                english_text = text  # Already English, copy as-is
            else:
                english_text = translate_to_english_chunked(text)  # Kiswahili -> English
            with open(translation_path, "w") as f:
                f.write(english_text)

            # === STAGE 4: English Audio Generation ===
            text_to_english_audio(english_text, english_audio_path)
            # Generates MP3 from English text using Google TTS

            # Mark complete
            state[pid]["processing_status"] = "completed"
            save_participants(state)

        time.sleep(5)  # Check again in 5 seconds
```

---

9. AUDIO PREPROCESSING — `audio_preprocess.py` (DeepFilterNet Noise Removal)

Three-Step Pipeline

```
Raw Call WAV ---> Step 1: FFmpeg (resample) ---> Step 2: DeepFilterNet (denoise) ---> Step
```

Step 1: Prepare Audio for Model

```
def _prepare_for_model(src_path, prep_path, sample_rate=48000, channel_mode="mixdown"):
    """Convert raw call audio to format DeepFilterNet needs"""

    # FFmpeg command: convert to 48kHz, mono, PCM 16-bit
    cmd = ["ffmpeg", "-y", "-i", src_path, "-ac", "1", "-ar", "48000", "-c:a", "pcm_s16le",
    subprocess.run(cmd)

    # Why 48kHz? DeepFilterNet was trained on 48kHz audio
    # Why mono? Phone calls are mono; mixing down stereo prevents issues
```

Step 2: DeepFilterNet Noise Removal (PyTorch)

```
def _enhance_with_deepfilternet(input_path, output_path):
    """Run PyTorch deep learning model to remove background noise"""

    # Load model (cached globally - only loaded once)
    (enhance_fn, model), df_state = _load_df_model()

    # Read WAV as numpy array, convert to PyTorch tensor
    with wave.open(input_path, "rb") as wf:
        frames = wf.readframes(wf.getnframes())
    arr = np.frombuffer(frames, dtype=np.int16).astype(np.float32) / 32768.0
    # Dividing by 32768 normalizes 16-bit audio to [-1.0, 1.0] range

    audio_tensor = torch.from_numpy(arr).unsqueeze(0)  # Add batch dimension

    # Run through DeepFilterNet model
    enhanced = enhance_fn(model, df_state, audio_tensor)

    # Write cleaned audio back to WAV
    pcm = enhanced.detach().cpu().squeeze(0).clamp(-1.0, 1.0).numpy()
    pcm = (pcm * 32767.0).astype(np.int16)  # Convert back to 16-bit
```

Step 3: Resample for Whisper

```
# FFmpeg: resample from 48kHz to 16kHz (Whisper needs 16kHz)
cmd = ["ffmpeg", "-y", "-i", enhanced_path, "-ac", "1", "-ar", "16000", "-c:a", "pcm_s16le"
subprocess.run(cmd)
```

Full Pipeline Function

```
def preprocess_recording(src_path, progress_cb=None):
    """Complete preprocessing pipeline"""

    cfg = _load_config()  # Read audio_processing section from config.yaml

    if not cfg.get("enabled"):
        return src_path  # Skip if disabled

    # Step 1: Prepare (48kHz mono)
    progress_cb(25, "Preparing audio for denoise")
    _prepare_for_model(src_path, prepared_path, model_sr=48000, channel_mode="mixdown")

    # Step 2: Denoise (DeepFilterNet)
    progress_cb(60, "Removing background noise")
    _enhance_with_deepfilternet(prepared_path, enhanced_path)

    # Step 3: Resample (16kHz for Whisper)
    progress_cb(75, "Resampling cleaned audio for Whisper")
    _run(["ffmpeg", "-y", "-i", enhanced_path, "-ac", "1", "-ar", "16000", final_path])

    # Cleanup intermediate files
    if not keep_intermediate:
        os.remove(prepared_path)
        os.remove(enhanced_path)

    return final_path  # data/audio_processed/participant_001.wav
```

---

10. WHISPER TRANSCRIPTION — `transcribe.py` (Speech-to-Text)

```
import whisper

# Load model ONCE at startup (cached globally)
# "large-v3" is the most accurate Whisper model
model = whisper.load_model("large-v3")

def transcribe_audio(file_path):
    """Transcribe audio file to text using Whisper"""

    result = model.transcribe(
        file_path,
        language="sw",                          # Force Kiswahili (don't auto-detect)
        task="transcribe",                      # Transcribe (not translate)
        fp16=False,                             # Use float32 (CPU safe, no GPU needed)
        verbose=False,                          # No progress output
        condition_on_previous_text=False,       # Prevents "hallucination drift"
        temperature=0.0                         # Deterministic (most confident prediction)
    )

    text = result["text"]                       # Kiswahili text
    detected_lang = result.get("language")      # Language Whisper thinks it heard
    return text, detected_lang
```

Key Parameters Explained: - `language="sw"` : Forces Kiswahili. Without this, Whisper might misdetect the language - `fp16=False` : Uses 32-bit float math. Needed for CPU; GPUs can use 16-bit for speed - `condition_on_previous_text=False` : Prevents the model from "hallucinating" — repeating phrases from earlier in the audio when it can't hear clearly - `temperature=0.0` : Greedy decoding — always picks the most likely word. Higher values add randomness

---

11. TRANSLATION — `translate.py` (Kiswahili to English)

```
from googletrans import Translator

translator = Translator()
MAX_CHARS = 3000  # Google Translate has a character limit per request

def _split_text(text, max_chars=3000):
    """Split text into chunks at sentence boundaries"""

    if len(text) <= max_chars:
        return [text]

    # Split on sentence-ending punctuation (.!?)
    sentences = re.split(r'(?<=[.!?])\s+', text)

    # Group sentences into chunks under 3000 characters
    chunks, current = [], ""
    for s in sentences:
        if len(current) + len(s) + 1 <= max_chars:
            current += " " + s
        else:
            chunks.append(current)
            current = s

    return chunks

def translate_to_english_chunked(text, retries=3, sleep_sec=1.5):
    """Translate long text with chunking and retry logic"""

    chunks = _split_text(text)
    out_chunks = []

    for idx, chunk in enumerate(chunks):
        for attempt in range(retries):
            try:
                res = translator.translate(chunk, src="sw", dest="en")
                out_chunks.append(res.text)
                break
            except Exception as e:
                time.sleep(sleep_sec)  # Wait before retry

        # If all retries failed, mark the chunk
        if len(out_chunks) < idx + 1:
            out_chunks.append(f"[TRANSLATION_FAILED_CHUNK {idx+1}]...")

    return "\n".join(out_chunks)
```

Why chunking? The googletrans library scrapes Google Translate, which has a ~5000 character limit per request. Using 3000 as a safe limit prevents failures.

---

12. TEXT-TO-SPEECH — `tts.py` (English Audio Generation)

```
from gtts import gTTS

def text_to_english_audio(text, out_path):
    """Convert English text to English MP3 audio"""

    text = text.strip()
    if not text:
        return False

    # Skip if translation failed (don't speak error markers)
    if "TRANSLATION_FAILED" in text:
        return False

    tts = gTTS(text=text, lang="en")
    tts.save(out_path)  # Saves as MP3
    return True
```

---

13. EXCEL EXPORT — `export_excel.py` (Survey Data Export)

How Responses Are Exported

```
def build_export_rows(state, participant_ids=None):
    """Build rows for Excel from participant responses"""

    metadata = _build_response_metadata()
    # metadata = {"q1": {"type": "open", "options": []},
    #             "q2": {"type": "mcq", "options": ["Opt A", "Opt B", "Opt C"]}, ...}

    for pid, pdata in state.items():
        responses = pdata.get("responses", {})

        # Filter: only MCQ/MCQO responses (no OPEN speech)
        filtered = filter_responses_for_excel(responses, metadata)

        row = {"participant_id": pid}
        for key, value in filtered.items():
            # Convert DTMF digit to option text
            # e.g., "2" -> "Kumaliza masomo" (the 2nd option)
            value = _decode_choice_value(value, rule["options"])
            row[export_key] = value

        rows.append(row)

def _decode_choice_value(value, options):
    """Convert stored digit to option text"""
    # value = "2", options = ["Kuhamia sehemu", "Kumaliza masomo", "Kufungua mkahawa"]
    digit = int(float(value))  # "2" -> 2
    idx = digit - 1             # 2 -> 1 (0-indexed)
    return options[idx]         # "Kumaliza masomo"

def export_excel_in_english(source_path, output_path):
    """Translate Excel cells to English"""

    df = pd.read_excel(source_path)
    cache = {}  # Cache translations to avoid duplicate API calls

    for idx, col in work_items:
        df.at[idx, col] = _translate_cell_to_english(df.at[idx, col], cache)

    df.to_excel(output_path, index=False)
```

14. SURVEY QUESTION FORMAT — `questions.txt`

Format Specification

Each line in `data/questions.txt` follows this pipe-delimited format:

```
TYPE|Question Text|Option1|Option2|Option3...
```

Question Types

Type	Format	Response Collection	Example
INFO	`INFO|Text`	No response (instruction only)	`INFO|Karibu kwenye utafiti wetu...`
OPEN	`OPEN|Text`	Speech captured for 6 seconds	`OPEN|Tafadhali sema jina lako`
MCQ	`MCQ|Text|Opt1|Opt2|Opt3`	1 keypad digit press	`MCQ|Mado anataka nini? |Kuhamia|Kumaliza|Kufungua`
MCQO	`MCQO|Text|Opt1|Opt2|Nyingine`	Digit + speech if "Other"	`MCQO|Kupanga watoto...|Ndiyo|Hapana|Nyingine`

How Questions Flow in a Call

```
questions[0] = INFO  -> Play intro (no response)
questions[1] = INFO  -> Play intro (no response)
questions[2] = OPEN  -> First real question (speech)
questions[3] = MCQ   -> Keypad press
questions[4] = MCQO  -> Keypad press (+ speech if "Other")
questions[5] = INFO  -> Section divider
questions[6] = MCQ   -> Continue survey...
...
questions[75] = MCQ  -> Last question
-> "Kwaheri" (Goodbye) -> Hangup
```

---

15. COMPLETE END-TO-END DATA FLOW

```
PHASE 1: SCHEDULING
  Admin uploads CSV with phone numbers via dashboard
  Admin sets scheduled time for each participant
  Participants stored in data/state/participants.json

PHASE 2: CALLING
  Scheduler thread checks every 15 seconds
  For each eligible participant: Twilio places call
  Participant answers -> Twilio sends webhook to /voice

PHASE 3: IVR SURVEY
  /voice: Start recording, redirect to /start
  /start: Play 2 intros, first question
  /next: Loop through all 76 questions
    OPEN: <Gather speech> -> store Kiswahili text
    MCQ:  <Gather dtmf>   -> store digit ("1", "2", "3")
    MCQO: <Gather dtmf>   -> store digit + optional speech
    INFO: <Play> -> auto-advance
  Survey done: "Kwaheri" -> Hangup

PHASE 4: RECORDING DOWNLOAD
  Twilio calls /recording-done webhook
  Download full call WAV from Twilio servers
  Save to data/audio/participant_001_20260321_143000.wav
  Set processing_status = "pending"

PHASE 5: ML PROCESSING (Background Worker)
  Step 1: DeepFilterNet noise removal
    data/audio/*.wav -> FFmpeg (48kHz) -> PyTorch denoise -> FFmpeg (16kHz)
    Output: data/audio_processed/*.wav

  Step 2: Whisper large-v3 transcription
    data/audio_processed/*.wav -> Kiswahili text
    Output: data/transcripts/*.txt

  Step 3: Google Translate
    Kiswahili text -> English text
    Output: data/translations/*.txt

  Step 4: Google TTS
    English text -> English MP3 audio
    Output: data/english_audio/*.mp3

PHASE 6: DATA EXPORT
  Admin clicks "Export Excel" on dashboard
  MCQ/MCQO responses extracted from participants.json
  Digits decoded to option text (e.g., "2" -> "Kumaliza masomo")
  Output: data/results/ivr_responses.xlsx
  Optional: Translate to English -> ivr_responses_english.xlsx
```

16. THREAD ARCHITECTURE & CONCURRENCY

```
Main Process (Python)
|
|-- Main Thread (Flask WSGI Server)
|     |-- Handles all HTTP requests (/voice, /start, /next, /admin, etc.)
|     |-- Runs on port 5050
|
|-- Scheduler Thread (daemon)
|     |-- Polls every 15 seconds
|     |-- Checks can_call() for each participant
|     |-- Places Twilio calls for eligible participants
|
|-- ML Worker Thread (daemon)
|     |-- Polls every 5 seconds
|     |-- Processes recordings: denoise -> transcribe -> translate -> TTS
|
|-- All threads share STATE_IO_LOCK (RLock)
|     |-- Prevents concurrent writes to participants.json
|     |-- Atomic writes via temp file + os.replace()
```

Why daemon threads? Daemon threads automatically die when the main process exits. No cleanup needed.

Why RLock (not Lock)? RLock allows the same thread to acquire the lock multiple times (reentrant). This prevents deadlock when `save_participants()` is called from within code that already holds the lock.

---

17. CONFIGURATION FILES

config.yaml

```
twilio:
  account_sid: ""           # Overridden by .env
  auth_token: ""            # Overridden by .env
  from_number: ""           # Overridden by .env

ivr:
  questions_file: "data/questions.txt"
  gather_timeout_sec: 6     # Seconds to wait for speech/keypad
  speech_timeout: "auto"    # Auto-detect end of speech
  speech_language: "sw-KE"  # Kiswahili (Kenya)
  recording_max_seconds: 1800  # 30 minutes

audio_processing:
  enabled: true
  backend: "deepfilternet"
  processed_dir: "data/audio_processed"
  model_sample_rate: 48000  # DeepFilterNet needs 48kHz
  output_sample_rate: 16000 # Whisper needs 16kHz
  channel_mode: "mixdown"   # Mix stereo to mono

auth:
  users:
    krishnanand:
      password_hash: "pbkdf2:sha256:1000000$..."
    professor:
      password_hash: "pbkdf2:sha256:1000000$..."
```

.env (Environment Variables)

```
TWILIO_ACCOUNT_SID=ACf2d28a...
TWILIO_AUTH_TOKEN=21a1b082...
TWILIO_FROM_NUMBER=+12764458808
PUBLIC_BASE_URL=https://armlike-unadduced-ai.ngrok-free.dev

AZURE_SPEECH_KEY=CGLZa3AA0pOXwnVR...
AZURE_SPEECH_REGION=eastus
AZURE_TTS_VOICE_SW="sw-KE-ZuriNeural"
AZURE_TTS_VOICE_EN="en-US-JennyNeural"
```

---

18. SECURITY FEATURES

Feature	Implementation
Password Storage	PBKDF2 with SHA-256, 1 million iterations (slow hash = brute-force resistant)
Brute-Force Protection	7 failures in 10 min = 15 min lockout, tracked by username+IP
Session Security	HTTPS-only cookies, HttpOnly (no JS access), SameSite=Lax (CSRF protection)
Session Lifetime	8 hours maximum
Audit Logging	All login/logout events logged to `auth_log.jsonl`
Route Protection	`/admin/*` routes require login; Twilio webhook routes are public
Atomic File Writes	Temp file + `os.replace()` prevents data corruption
Phone Masking	Phone numbers masked in logs: `+1******7890`

19. KEY DESIGN DECISIONS & WHY

Decision	Why
File-based storage (no database)	Simplicity, portability, no setup needed. JSON files are easy to inspect and backup.
Daemon threads (not processes)	Simpler than multiprocessing, shares memory, auto-cleanup on exit.
Hybrid speech approach	Twilio `<Gather>` for live IVR (real-time), Whisper for post-call (high accuracy). Live transcription doesn't need to be perfect.
Text hash caching for TTS	Azure TTS costs money per character. Caching by SHA1 hash means same text is only synthesized once.
Chunked translation	Google Translate has character limits. 3000-char chunks with retry prevents failures.
Configurable surveys	Change `questions.txt` to deploy a different survey without any code changes.
Atomic writes	Write to temp file, then `os.replace()`. If the app crashes mid-write, the original file is intact.
Max 3 attempts per participant	Respects participant's time. If they don't answer 3 times, stop calling.
1-hour retry gap	Don't spam-call someone who just didn't answer.
15% slower TTS prosody	Kiswahili Neural TTS speaks slightly too fast by default. Slowing by 15% improves comprehension.

20. LIBRARIES & WHAT EACH ONE DOES

Direct Dependencies

Library	Version	Purpose
`flask`	3.1.2	Web framework — handles HTTP routes, sessions, templates
`twilio`	9.10.0	Twilio API client — places calls, generates TwiML
`openai-whisper`	20240930	Speech-to-text AI model — transcribes Kiswahili audio
`azure-cognitiveservices-speech`	1.48.1	Azure Neural TTS — generates Kiswahili voice prompts
`googletrans`	4.0.0rc1	Google Translate wrapper — translates Kiswahili to English
`gtts`	2.5.4	Google TTS — generates English audio from text
`deepfilternet`	0.5.6	Neural noise removal — cleans call recordings
`torch`	2.8.0	PyTorch ML framework — powers DeepFilterNet
`torchaudio`	2.8.0	Audio processing for PyTorch
`numpy`	1.26.4	Numerical computing — audio data as arrays
`pandas`	2.3.3	Data manipulation — builds Excel export DataFrames
`openpyxl`	3.1.5	Excel file generation — writes .xlsx files
`pyyaml`	6.0.3	YAML parser — reads config.yaml
`python-dotenv`	1.2.1	Environment variables — loads .env file
`werkzeug`	(Flask dep)	Password hashing — PBKDF2 for auth
`colorlog`	6.10.1	Colored console output — prettier logs
`requests`	(dep)	HTTP client — downloads recordings from Twilio
`pydub`	0.25.1	Audio manipulation — format conversion

21. GLOSSARY

Term	Meaning
IVR	Interactive Voice Response — automated phone system that interacts with callers
TwiML	Twilio Markup Language — XML format for scripting phone call behavior
DTMF	Dual-Tone Multi-Frequency — the technical name for phone keypad tones
TTS	Text-to-Speech — converting written text into spoken audio
STT	Speech-to-Text — converting spoken audio into written text
SSML	Speech Synthesis Markup Language — XML format for controlling TTS voice
Webhook	An HTTP callback — a URL that a service (Twilio) calls to notify your app of events
ngrok	A tunnel service that exposes local servers to the internet via HTTPS
PBKDF2	Password-Based Key Derivation Function 2 — slow hashing algorithm for passwords
RLock	Reentrant Lock — a thread lock that the same thread can acquire multiple times
Daemon Thread	A background thread that automatically dies when the main program exits
E.164	International phone number format (e.g., +12764458808)
PCM	Pulse-Code Modulation — raw uncompressed audio format
WAV	Waveform Audio File Format — uncompressed audio container
MP3	Compressed audio format
DeepFilterNet	A neural network for real-time speech enhancement (noise removal)
Whisper	OpenAI's speech recognition model (supports 99 languages)
Flask Blueprint	A way to organize Flask routes into separate modules
Atomic Write	Writing to a temp file then renaming — prevents partial/corrupted writes

22. PRESENTATION TALKING POINTS

Opening

"AudioSurvey AI is a platform that automates voice-based research surveys in Kiswahili for African refugee populations using AI."

Problem Statement

"Traditional surveys require in-person enumerators, which is expensive and hard to scale. Our system automates the entire process over phone calls."

Technical Highlights

1. "We use **Twilio** to place automated phone calls and an **IVR system** to ask 76 survey questions in Kiswahili"
2. "Survey questions are spoken using **Azure Neural TTS** with Kiswahili voice `sw-KE-ZuriNeural`"
3. "Responses are collected via **speech recognition** (for open-ended) and **keypad presses** (for multiple choice)"
4. "After the call, recordings go through our **4-stage ML pipeline**: noise removal (DeepFilterNet), transcription (Whisper), translation (Google Translate), and English audio generation (gTTS)"
5. "Everything is managed through a **web-based admin dashboard** with authentication and real-time monitoring"
6. "The entire system is **configurable** — change the survey by editing a text file, no code changes needed"

Architecture Summary

"Flask web server with 3 threads: main thread handles HTTP, scheduler thread places calls every 15 seconds, and ML worker thread processes recordings. All data is stored in JSON files with thread-safe atomic writes."

End of Study Guide